

《FPGA》考试试卷参考答案

一、单项选择题（2×15=30 分）

1. 下列哪种数据类型主要用于组合逻辑建模(**B**)。 A. reg B. wire C. integer D. real
2. 以下哪个符号表示按位或运算(**D**)。 A. && B. || C. & D. |
3. 在 Verilog 中,定义一个 8 位宽的寄存器应使用(**A**)。 A. reg[7:0] var B. reg[0:7] var C. reg[8:1] var
4. 阻塞赋值符号为 _____,非阻塞赋值符号为 _____。(**B**) A. <= = B. = <= C. = ==
5. 在 always 块中,敏感列表使用“@(*)”表示(**C**)。 A. 电平敏感 B. 边沿敏感 C. 所有输入信号
6. 下列语句中不属于并行语句的是(**B**)。
注:always 块内部的 case 语句属于顺序语句。 A. 过程语句 B. Case 语句 C. assign 语句 D. 连续赋值语句
7. 在 Verilog 中,“parameter”用于定义(**B**)。 A. 变量 B. 常量 C. 模块实例 D. 函数
8. 下列哪个不是 Verilog 的循环语句(**D**)。 A. for B. while C. repeat D. loop
9. 在 Verilog 中,4'b1011 » 1 的结果是(**A**)。 A. 4'b0101 B. 4'b1101 C. 4'b0110 D. 4'b1010
10. 下列哪个是 Verilog 中的条件语句(**D**)。 A. if-else B. case C. switch D. A 和 B 都是

二、填空题（每空 1 分，共 15 分）

11. 在 Verilog 中,定义一个 8 位宽的输入端口应写作 _____ **input** _____ [7:0] signal_name。
12. 表达式 {3{2'b10}} 的位拼接结果是(二进制表示) _____ **101010** _____。
13. 时序逻辑中的 always 块应使用 _____ **边沿** _____ 触发(填写“电平”或“边沿”)。
14. IEEE 工业标准的硬件描述语言有 _____ **Verilog** _____ 和 _____ **VHDL** _____ 两种。
15. 表达式 4'b1011 & 4'b0011 的运算结果是(二进制表示) _____ **0011** _____。
16. 在测试模块中,生成周期为 10 个时间单位的时钟信号应使用:
_____ **always** _____ (填写关键字) #5 clk = _____ **~clk** _____; (补全语句)
17. 模块实例化时,端口连接方式分为 _____ **命名(或名称)** _____ 连接和顺序连接两种。
18. 表达式 ~(4'b0011) 的逻辑运算结果是(二进制表示) _____ **1100** _____。

19. 定义有符号数寄存器应使用 reg signed [7:0] data; (填写关键字)
20. 在 case 语句中, default 分支可以匹配所有未明确指定的情况。
21. 下面两段代码中信号 in, q1, q2 和 q3 的初值分别为 0, 1, 2 和 3, 那么经过 1 个时钟周期后, 左侧程序中 q3 的值变成 0, 右侧程序中 q3 的值变成 2。
22. 我们上课使用的开发板中主控芯片为 xilinx zynq 7020 (填型号)。

三、简答题 (共 35 分)

23. 在 FPGA 设计中, Verilog 的 always@(*) 语句和 always@(posedge clk) 语句分别用于描述什么类型的逻辑电路? 它们对应的实际硬件结构有何不同? (10 分)

答:

(1) always@(*) 用于描述组合逻辑电路。其实际硬件结构对应的是由基本逻辑门(与、或、非等)和多路选择器构成的无状态记忆的纯逻辑网络。

(2) always@(posedge clk) 用于描述时序逻辑电路。其实际硬件结构对应的是包含触发器(如 D 触发器)或寄存器的具有状态记忆功能的电路, 输出受时钟边沿驱动。

24. 简述 FPGA 设计开发流程, 并简要介绍每一步都具体完成什么功能。(10 分)

答: 标准的 FPGA 开发流程主要包含以下几步:

(1) 设计输入: 通过编写 Verilog/VHDL 代码或绘制原理图, 完成系统逻辑功能的设计。

(2) 综合 (Synthesis): 将高级硬件描述语言转换并映射为底层逻辑门和触发器组成的门级网表。

(3) 实现 (Implementation): 包含布局 (Placement) 和布线 (Routing), 将网表中的逻辑门映射到 FPGA 内部的具体物理资源(如 LUT、FF), 并连线。

(4) 仿真 (Simulation): 包括综合前仿真(功能/RTL 仿真)验证逻辑功能, 以及后仿真(时序仿真)验证加上硬件延迟后的时序是否满足要求。

(5) 配置下载: 生成最终的比特流文件 (Bitstream) 并烧录或下载到 FPGA 芯片中进行上板验证。

25. 请根据以下内容, 完成下面的运算(用二进制表示)(15 分)

(1) wire[3:0] A,B; 且 $A=4'b1100$, $B=4'b1010$, 求 $A \& B = 1000$

(2) wire[3:0] A; 且 $A=4'b0110$, 求 $\sim A = 1001$

(3) wire[3:0] A,B; 且 $A=4'b1011$, $B=4'b0110$, 求 $A \wedge B = 1101$

(4) wire[3:0] A; 且 $A=4'b1001$, 求 $A \gg 2 = 0010$

- (5) wire signed[3:0] A; 且 $A=4'b1011$ (-5), 求 $A \gg 1 = \mathbf{1101}$
- (6) wire[3:0] A; 且 $A=4'b1100$, 求 $\{A[1:0], A[3:2]\} = \mathbf{0011}$
- (7) wire[3:0] A,B; 且 $A=4'b1010$, $B=4'b0101$, 求 $A + B = \mathbf{1111}$
- (8) wire signed[3:0] A,B; 且 $A=4'b1100$ (-4), $B=4'b0010$ (+2), 求 $A - B = \mathbf{1010}$
- (9) wire[3:0] A,B; 且 $A=4'b0011$, $B=4'b0100$, 求 $A * 2 + B = \mathbf{1010}$
- (10) wire[3:0] A,B; 且 $A=4'b1010$, $B=4'b1001$, 求 $(A > B) = \mathbf{1}$ (或 $1'b1$)
- (11) wire signed [3:0] A,B; 且 $A=4'b1101$ (-3), $B=4'b1110$ (-2), 求 $(A == B) = \mathbf{0}$ (或 $1'b0$)
- (12) wire[3:0] A,B; 且 $A=4'b1100$, $B=4'b0011$, 求 $(A | B) \& (A \wedge B) = \mathbf{1111}$
- (13) wire[3:0] A,B; 且 $A=4'b1010$, $B=4'b0101$, 求 $(A > B) ? (A << 1) : (B >> 1) = \mathbf{0100}$
- (14) wire signed[3:0] A,B; 且 $A=4'b1001$ (-7), 执行 $A = ((A << 1) + 1'b1)$ 后, $A = \mathbf{0011}$
- (15) wire[3:0] A,B,C; 且 $A=4'b0011$, $B=4'b0100$, $C=4'b1000$, 求 $(A + B) \& C = \mathbf{0000}$

四、程序设计题（共 40 分）

26. 如图所示模块，设计一个带复位端和时钟的三个 8bit 输入信号（无符号）的比较器，输出最小值。（10 分）

```

1 module compare_module(
2     input clk,
3     input rst_n,
4     input [7:0] a,
5     input [7:0] b,
6     input [7:0] c,
7     output reg [7:0] d
8 );
9     always @(posedge clk or negedge rst_n) begin
10         if (!rst_n) begin
11             d <= 8'd0;
12         end else begin
13             if (a <= b && a <= c)
14                 d <= a;
15             else if (b <= a && b <= c)

```

```

16         d <= b;
17     else
18         d <= c;
19     end
20 end
21 endmodule

```

27. 试设计一个带复位的 3/8 译码器,规定模块定义为 module Decoder(Out,In,En)。(10 分)

```

1 module Decoder(
2     output reg [7:0] Out,
3     input [2:0] In,
4     input En
5 );
6     always @(*) begin
7         if (!En) begin
8             Out = 8'b0000_0000;
9         end else begin
10            case (In)
11                3'b000: Out = 8'b0000_0001;
12                3'b001: Out = 8'b0000_0010;
13                3'b010: Out = 8'b0000_0100;
14                3'b011: Out = 8'b0000_1000;
15                3'b100: Out = 8'b0001_0000;
16                3'b101: Out = 8'b0010_0000;
17                3'b110: Out = 8'b0100_0000;
18                3'b111: Out = 8'b1000_0000;
19                default: Out = 8'b0000_0000;
20            endcase
21        end
22    end
23 endmodule

```

28. 设计一个同时输出 2/4/8 分频的时钟分频器,并编写 testbench 文件,画出输入输出信号波形。(20 分)

```

1 // RTL 设计模块
2 module div_module(

```

```
3     input rst,
4     input clk_in,
5     output clk_out2,
6     output clk_out4,
7     output clk_out8
8 );
9 // 使用一个3位计数器实现分频
10 reg [2:0] cnt;
11
12 always @(posedge clk_in or negedge rst) begin
13     if (!rst)
14         cnt <= 3'd0;
15     else
16         cnt <= cnt + 1'b1;
17 end
18
19 // 计数器的第0、1、2位分别对应2、4、8分频
20 assign clk_out2 = cnt[0];
21 assign clk_out4 = cnt[1];
22 assign clk_out8 = cnt[2];
23 endmodule
24
25 // Testbench 测试文件
26 `timescale 1ns/1ps
27 module tb_div_module();
28     reg clk_in;
29     reg rst;
30     wire clk_out2;
31     wire clk_out4;
32     wire clk_out8;
33
34     div_module u_div (
35         .rst(rst),
36         .clk_in(clk_in),
37         .clk_out2(clk_out2),
38         .clk_out4(clk_out4),
39         .clk_out8(clk_out8)
40     );
41
42 // 产生周期为 10ns 的时钟
```

```
43     initial begin
44         clk_in = 0;
45         forever #5 clk_in = ~clk_in;
46     end
47
48     // 产生复位信号
49     initial begin
50         rst = 0;
51         #15 rst = 1;
52         #300 $finish;
53     end
54 endmodule
```

简要波形说明（复位结束后）：

clk_in: 周期信号（基础频率）

clk_out2: 每过 2 个 clk_in 边沿翻转一次，周期为 clk_in 的 2 倍。

clk_out4: 每过 4 个 clk_in 边沿翻转一次，周期为 clk_in 的 4 倍。

clk_out8: 每过 8 个 clk_in 边沿翻转一次，周期为 clk_in 的 8 倍。

波形示意图：

